

ANSIBLE

IN A NUTSHELL



From First Playbook to Production Automation

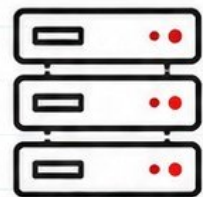
**SIMPLE.
POWERFUL.
AUTOMATION.**



ANSIBLE
CONTROL NODE



PLAYBOOK



MANAGED NODES
(SERVERS)



CONFIGURE
SERVERS



DEPLOY
APPLICATIONS



MANAGE
CONFIGURATION



AUTOMATE
AT SCALE



IMPROVE
RELIABILITY

- ✓ STEP-BY-STEP LEARNING FOR BEGINNERS
- ✓ REAL-WORLD EXAMPLES
- ✓ PRODUCTION BEST PRACTICES
- ✓ TROUBLESHOOTING GUIDE
- ✓ BUILD PRACTICAL SKILLS

**LESS
MANUAL WORK
MORE
IMPACT**

**INFRASTRUCTURE
AUTOMATION
FOR A BETTER TOMORROW.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN

| PRACTICE

| AUTOMATE

| GROW

ANSIBLE EXPLAINED, WHY ENGINEERS AUTOMATE



AUTOMATE TODAY. MORE TIME FOR WHAT MATTERS.

SIMPLE
POWERFUL
AGENTLESS

1 WHAT ANSIBLE IS



Ansible is an open source automation tool used to configure, deploy and manage infrastructure.

- Agentless
- Simple YAML syntax
- Uses SSH (no extra software on servers)

2 THE INFRASTRUCTURE PROBLEM IT SOLVES



Managing multiple servers manually is slow, error-prone and inconsistent.

- Every server can be different
- Hard to reproduce changes
- Small mistakes can cause big problems
- Difficult to scale

3 MANUAL ADMINISTRATION vs AUTOMATION

MANUAL ADMINISTRATION

- ✗ Log in to each server
- ✗ Run commands one by one
- ✗ Inconsistent configuration
- ✗ Prone to human error
- ✗ Not scalable

AUTOMATION WITH ANSIBLE

- ✓ Define changes in playbooks
- ✓ Run on many servers at once
- ✓ Consistent and repeatable
- ✓ Fewer errors
- ✓ Easy to scale

4 CONFIGURATION MANAGEMENT



Ansible helps you keep your infrastructure in the desired state.

- Install and configure software
- Manage configuration files
- Ensure services are running
- Apply the same configuration across environments

5 REPEATABILITY AND CONSISTENCY



You write the automation once, then run it as many times as needed.

- Same result every time
- Eliminates configuration drift
- Easy to reproduce environments
- Builds reliable infrastructure

6 WHERE ANSIBLE FITS IN DEVOPS



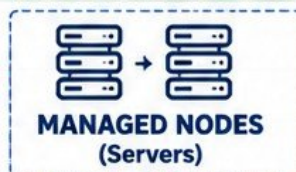
Ansible is used across the DevOps lifecycle for infrastructure automation.

- Provision and configure servers
- Deploy applications
- Manage cloud infrastructure
- Integrate with CI/CD pipelines
- Support configuration management and infrastructure as code (IaC)

7 MENTAL MODEL: CONTROL NODE → SSH → MANAGED NODES



SSH



You run Ansible from a control node (your laptop or a server). It connects to managed nodes using SSH and executes the tasks defined in your playbooks.

8 JUNIOR ENGINEER TIP



Automate what you understand first.

Start with simple tasks, learn how they work, then gradually automate more complex infrastructure.

UNDERSTAND
AUTOMATE
IMPROVE

SAME
SKILLS
BIGGER
OPPORTUNITIES



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

HOW ANSIBLE ACTUALLY WORKS

AUTOMATE
CONFIGURE
DEPLOY
SCALE

SIMPLE ARCHITECTURE. POWERFUL AUTOMATION.

1 AGENTLESS ARCHITECTURE



Ansible is agentless, which means you do not need to install any Ansible software on managed nodes. It uses SSH to connect and run modules remotely.

2 CONTROL NODE



- The control node is where you run Ansible commands and playbooks.
- It can be your local laptop or a central automation server.
- It contains Ansible, your inventory, playbooks, roles and configuration.

3 MANAGED NODES



- Managed nodes are the target servers you want to configure and manage.
- They are usually Linux or Unix systems.
- No Ansible software is installed.
- They must be reachable over SSH.
- Python is required on managed Linux hosts (already installed on most systems).

4 INVENTORY



- The inventory tells Ansible which servers to manage.
- You can define individual hosts, groups and variables.
- It can be a static file or a dynamic inventory from a cloud provider.

5 SSH



- Ansible uses SSH to connect to managed nodes.
- It copies small modules to the remote server and executes them.
- You need SSH access and the right permissions (usually a user with sudo access).

6 PYTHON ON MANAGED HOSTS



- Ansible modules run on the managed node using Python.
- Python is usually pre-installed on most Linux distributions.
- If it is missing, you may need to install it manually.

7 MODULES



- Modules are small programs that perform tasks on managed nodes.
- Examples: package, service, file, copy, user, template.
- Modules make tasks idempotent and safer than raw shell commands.

8 TASKS



- Tasks define what you want Ansible to do on the managed nodes.
- Each task uses a module.
- Tasks run in order, and you can add conditions, loops and variables.

9 PLAYBOOKS



- A playbook is a YAML file that contains one or more plays.
- It defines the target hosts, variables, tasks and handlers.
- Playbooks allow you to automate complex, repeatable infrastructure tasks.
- You run a playbook using: `ansible-playbook <file.yml>`

10 HOW IT ALL WORKS TOGETHER



SSH

Runs modules remotely



KEY TAKEAWAY



You write playbooks on the control node. Ansible connects to your servers over SSH, runs modules using Python, and performs the tasks.

AUTOMATION
BUILDS A STRONGER
TOMORROW.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN. PRACTICE. AUTOMATE. GROW.

INSTALL ANSIBLE AND RUN YOUR FIRST COMMAND

PRACTICE
BUILD
AUTOMATE
GROW

GET ANSIBLE RUNNING AND CONNECT TO YOUR FIRST SERVERS

1 INSTALLATION



Install Ansible using your system package manager. Examples:

```
# On Ubuntu / Debian
sudo apt update
sudo apt install ansible

# On RHEL / CentOS / Rocky
sudo dnf install ansible

# On macOS (Homebrew)
brew install ansible
```

2 CHECK ANSIBLE VERSION

Confirm Ansible is installed correctly.

```
ansible --version
```

You should see output similar to:

```
ansible [core 2.16.0]
  config file = None
  configured module search path = [...]
  python version = 3.11.6
  jinja version = 3.1.2
  libyaml = True
```

3 CONFIRMING CONFIGURATION

Check your Ansible configuration details.

```
ansible-config view
```

```
# Show the config file being used
ansible-config view --only-changed
```



If you do not have a config file yet, Ansible will use the default settings (this is fine for now).

4 CREATE A PROJECT DIRECTORY



Keep your Ansible files organized in a project directory.

```
mkdir -p ~/ansible-demo
cd ~/ansible-demo

# Create basic structure
mkdir inventory playbooks
```

5 YOUR FIRST INVENTORY



Create a simple inventory file with your server(s).

```
# inventory/hosts
[servers]
server1 ansible_host=192.168.1.10
server2 ansible_host=192.168.1.11
```

6 TESTING CONNECTIVITY

Test if Ansible can reach your servers.

```
ansible -i inventory/hosts \
  all -m ping
```



If you see "SUCCESS", Ansible can connect to your servers.

7 ansible all -m ping

This command uses the built-in ping module to test connectivity and Python on the managed nodes.

```
$ ansible -i inventory/hosts all -m ping

server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

8 WHAT ANSIBLE'S PING ACTUALLY TESTS

- ✓ Can Ansible connect to the server using SSH
- ✓ Can it authenticate with the user you provided
- ✓ Whether Python is available on the server (Ansible modules require Python)
- ✓ It runs a simple module and returns "pong"
- ✓ It does not test your application, services or full system configuration



A successful ping means Ansible can run modules on the server. It does not mean everything is configured yet.

9 COMMON FIRST-RUN FAILURES

- ✗ **SSH connection failed**
Wrong IP, hostname or network access.
- ✗ **Permission denied**
Incorrect user or missing SSH key.
- ✗ **Python not found**
Install Python on the managed node.
- ✗ **Host key checking failed**
Add the host to known_hosts or disable strict checking (not recommended for production).
- ✗ **Module failed to execute**
Check the error message with `-vvv` for more details.
- ✗ **Inventory not found**
Use the correct path with `-i`.

SMALL
STEPS
BIG RESULTS



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

INVENTORY

KNOW WHAT YOU ARE MANAGING

ORGANIZE
INVENTORY
CONTROL
AUTOMATE

YOUR INVENTORY DEFINES THE SERVERS ANSIBLE MANAGES

1 INVENTORY FILES



- The inventory file tells Ansible which servers to manage.
- It can be a static file or dynamically generated.
- It contains hosts, groups and variables.

2 HOSTS



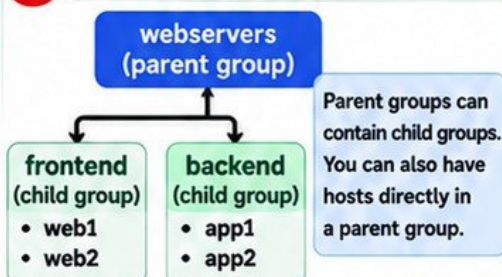
- A host is a server (physical or virtual) that Ansible manages.
- You can use a hostname or an IP address.
- Each host can have its own variables.

3 GROUPS



- Groups help you organize hosts.
- You can run commands on a single host or an entire group.
- Groups make it easier to manage large environments.

4 PARENT AND CHILD GROUPS



5 HOSTNAMES vs IP ADDRESSES

HOSTNAMES



- Easy to read
- Easier to manage
- Recommended for production

IP ADDRESSES



- Works even without DNS
- Useful in small environments
- Can be harder to maintain

6 INVENTORY VARIABLES



- Define variables for individual hosts or groups.
- Used for environment-specific configuration (e.g. database URLs, ports).
- Can be set in the inventory file, group_vars or host_vars directories.

7 STATIC INVENTORY



- A simple text file that you create and manage.
- Suitable for small to medium environments.
- Stores hosts, groups and variables.

```
# inventory/hosts
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20
```

8 DYNAMIC INVENTORY



- Inventory is generated automatically from a cloud provider or external source.
- Used in large or cloud environments (AWS, Azure, GCP, etc.).
- Always up to date with your infrastructure.

Examples of dynamic inventory sources:

- AWS EC2 (aws_ec2)
- Azure (azure_rm)
- Google Cloud (gcp_compute)
- VMware (vmware_vm_inventory)
- Custom scripts

9 ansible-inventory --list



- This command shows your inventory in JSON format. It helps you verify that your inventory is correct.

```
$ ansible-inventory --list
{
  "webservers": {
    "hosts": ["web1", "web2"]
  },
  "dbservers": {
    "hosts": ["db1"]
  }
}
```

10 PRODUCTION INVENTORY ORGANIZATION



- Use clear and consistent naming.
- Organize by environment (dev, staging, prod).
- Use group_vars and host_vars for variables.
- Keep inventory in version control (Git).
- Use dynamic inventory for cloud environments.
- Document your inventory structure.
- Follow least privilege access.
- Test changes in a non-production environment first.



A well-organized inventory makes your Ansible automation easier, safer and more scalable.

INFRASTRUCTURE
AS CODE
A BRIGHTER TOMORROW.



LEARN. PRACTICE. AUTOMATE. GROW.

AD HOC COMMANDS FOR FAST OPERATIONS

FAST
COMMANDS
REAL
RESULTS

GET THINGS DONE QUICKLY WITH ANSIBLE

1 WHAT AN AD HOC COMMAND IS



- An ad hoc command is a one-line Ansible command that runs a single task on one or more hosts.
- Useful for quick operations and troubleshooting.
- Does not require a playbook.

2 COMMAND STRUCTURE

```
ansible <hosts> -m <module> -a "<arguments>"
```

- **ansible** : the command
- **<hosts>** : target hosts or groups
- **-m** : module to use
- **-a** : module arguments

3 HOST PATTERNS

- **all** : all hosts in inventory
- **web** : a specific group
- **server1** : a specific host
- **web*** : pattern matching
- **!db** : exclude a host or group
- **web:&prod** : intersection (web AND prod)

4 -m (MODULE)

- Specifies the Ansible module to run.
- Examples: command, shell, copy, file, package, service, etc.

Example:

```
ansible web -m ping
```

5 -a (ARGUMENTS)

- Passes the arguments to the module.
- The format depends on the module.
- Use quotes for multiple arguments or values with spaces.

Example:

```
ansible web -m command \
-a "uptime"
```

6 COMMON MODULES AND EXAMPLES

Module	What It Does	Example
command	Run a command (does not use shell)	ansible all -m command -a "uptime"
shell	Run a shell command (with shell features)	ansible all -m shell -a "ls -l /tmp"
copy	Copy files to remote hosts	ansible web -m copy -a "src=file.txt dest=/tmp/"
file	Manage files and directories	ansible web -m file -a "path=/tmp/test state=directory"
package	Install or remove packages	ansible web -m package -a "name=nginx state=present"

7 EXAMPLES IN ACTION

```
# Check connectivity (ping module)
ansible all -m ping
```

```
# Run a command
ansible web -m command -a "uptime"
```

```
# Run a shell command
ansible web -m shell -a "df -h"
```

```
# Copy a file
ansible web -m copy -a "src=index.html
dest=/var/www/html/"
```

```
# Create a directory
ansible web -m file -a "path=/opt/app
state=directory"
```

```
# Install a package
ansible web -m package -a "name=nginx
state=present"
```

8 WHEN AD HOC COMMANDS ARE USEFUL



- Quick checks (connectivity, disk space, uptime)
- One-off tasks
- Troubleshooting
- Testing modules
- Small changes
- Learning and experimentation

9 WHY THEY SHOULD NOT REPLACE REPEATABLE PLAYBOOKS



- Not version controlled
- Hard to repeat complex tasks
- Not suitable for multi-step workflows
- Difficult to maintain at scale
- No structure, error handling or reusability
- Playbooks are better for consistent, repeatable and production-ready automation

10 KEY TAKEAWAYS



- Ad hoc commands are powerful for quick operations.
- Use the correct module for the task.
- Understand host patterns.
- They are great for learning and troubleshooting.
- Use playbooks for anything complex or repeatable.
- Start with ad hoc commands, but grow into playbooks.

QUICK
TASKS TODAY.
BIGGER
AUTOMATION
TOMORROW.

PRACTICE TODAY
AUTOMATE TOMORROW
WITH VERIQTA.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN. PRACTICE. AUTOMATE. GROW.

YOUR FIRST ANSIBLE PLAYBOOK

WRITE
AUTOMATE
REPEAT
GROW

SIMPLE YAML. POWERFUL AUTOMATION.

1 WHAT A PLAYBOOK IS



- A playbook is a YAML file that contains one or more plays.
- It defines what you want to do, on which servers, and how to do it.
- Playbooks allow you to automate repeatable tasks, configuration and deployments.

2 YAML STRUCTURE



- Ansible playbooks are written in YAML (YAML Ain't Markup Language).
- YAML uses indentation (spaces, not tabs).
- It is human readable and easy to write.

3 PLAYS



- A play targets a group of hosts and defines a set of tasks.
- A play can have variables, handlers and more.
- A playbook can contain one or multiple plays.

4 HOSTS



- The hosts keyword defines which servers the play will run on.
- You can use a group name, a single host or all.
- Examples: all, web, db, localhost.

Example:

```
hosts: webservers
```

5 BECOME



- The become keyword allows you to run tasks with elevated privileges (e.g., sudo).
- Commonly used for administrative tasks like installing packages or modifying system files.

Example:

```
become: true
```

6 TASKS



- Tasks define the individual actions to be performed on the target hosts.
- Each task uses a module.
- Tasks run in order.
- You can add conditions, loops and variables.

7 MODULES



- Modules are the building blocks of Ansible.
- They perform tasks on the managed nodes.
- Examples: package, service, copy, file, user, command, shell.
- Use fully qualified collection names (FQCN) when required.

8 TASK NAMES



- Give each task a meaningful name using the name keyword.
- Task names make your playbook easier to read and troubleshoot.

Example:

```
- name: Install Nginx
  ansible.builtin.package:
    name: nginx
    state: present
```

9 YOUR FIRST PLAYBOOK EXAMPLE

```
# file: first-playbook.yml
---
- name: Install and start Nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx
      ansible.builtin.package:
        name: nginx
        state: present
    - name: Start and enable Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

10 RUNNING THE PLAYBOOK



- Use the `ansible-playbook` command to run your playbook.
- Specify the file name.
- You can also use `-i` to specify a different inventory file.

Example:

```
ansible-playbook -i inventory/hosts
first-playbook.yml
```



If everything is correct, Ansible will run the tasks on the target hosts.

11 READING PLAY, TASK AND RECAP OUTPUT

```
PLAY [Install and start Nginx] *****
TASK [Gathering Facts] ***** ok: [web1]
TASK [Install Nginx] ***** changed: [web1]
TASK [Start and enable Nginx] ***** changed: [web1]
PLAY RECAP *****
web1 : ok=3 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

- **PLAY:** shows the name of the play being executed.
- **TASK:** shows each task and its result.
- **ok:** the task ran successfully, no changes.
- **changed:** the task made a change.
- **failed:** the task failed.
- **unreachable:** Ansible could not connect to the host.
- **RECAP:** a summary of results for each host.

AUTOMATE
LEARN
BUILD
A BETTER TOMORROW.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN. PRACTICE. AUTOMATE. GROW.

MODULES

THE BUILDING BLOCKS OF ANSIBLE

SMALL
MODULES
BIG IMPACT

SIMPLE MODULES. POWERFUL AUTOMATION.

1 WHAT MODULES DO



- Modules are small, reusable programs that perform specific tasks on managed nodes.
- They handle the complexity for you.
- Modules make your playbooks idempotent, reliable and easier to maintain.

2 FULLY QUALIFIED COLLECTION NAMES (FQCN)



- Ansible modules are identified using a Fully Qualified Collection Name (FQCN).
- Format: collection_name.module_name
- Example: `ansible.builtin.package`
- Using FQCN avoids confusion and ensures you use the correct module.

3 `ansible.builtin`



- `ansible.builtin` is the default collection that comes with Ansible.
- It contains the core modules used in most playbooks.
- You can usually just specify the module name (e.g. `package`) but using the full name (`ansible.builtin.package`) is best practice.

4 COMMON MODULES AND EXAMPLES

Here are some of the most commonly used Ansible modules:



package

Install or remove packages on the managed node.

```
- name: Install Nginx
  ansible.builtin.package:
    name: nginx
    state: present
```



service

Manage services (start, stop, enable).

```
- name: Start and enable Nginx
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```



user

Manage user accounts.

```
- name: Create a user
  ansible.builtin.user:
    name: devops
    state: present
    shell: /bin/bash
```



file

Manage files and directories.

```
- name: Create a directory
  ansible.builtin.file:
    path: /opt/app
    state: directory
    mode: '0755'
```



copy

Copy files from control node to managed node.

```
- name: Copy configuration
  ansible.builtin.copy:
    src: nginx.conf
    dest: /etc/nginx/nginx.conf
    mode: '0644'
```



template

Create files from templates (using Jinja2).

```
- name: Deploy configuration
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/app.conf
    mode: '0644'
```



uri

Interact with HTTP/HTTPS APIs.

```
- name: Call an API
  ansible.builtin.uri:
    url: https://api.example.com
    method: GET
    status_code: 200
    return_content: true
```



debug

Display messages and variable values.

```
- name: Show a message
  ansible.builtin.debug:
    msg: "Deployment completed"

- name: Show a variable
  ansible.builtin.debug:
    var: ansible_hostname
```

5 FINDING MODULE DOCUMENTATION



- Use the Ansible documentation website: <https://docs.ansible.com>
- Search by module name (e.g. `package`).
- You can also use the `ansible-doc` command:

```
ansible-doc ansible.builtin.package
```

6 WHY MODULES ARE USUALLY SAFER THAN SHELL COMMANDS



- ✓ Modules are idempotent (safe to run repeatedly).
- ✓ They handle errors and edge cases.
- ✓ They provide consistent behavior across systems.
- ✓ They are easier to read and maintain.
- ✓ They reduce the risk of accidental changes.
- ✓ Use shell only when a module does not exist for the task.



KEY TAKEAWAY

Modules are the building blocks of Ansible. Use the right module, read the documentation, and prefer modules over shell commands for safer, more reliable and repeatable automation.

PRACTICE TODAY
AUTOMATE TOMORROW
WITH VERIQTA.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN. PRACTICE. AUTOMATE. GROW.

IDEMPOTENCY,

ANSIBLE'S MOST IMPORTANT MENTAL MODEL

SAME RESULT. EVERY TIME.

1 WHAT IDEMPOTENCY MEANS



Idempotency means Ansible brings your infrastructure to a desired state and will not make unnecessary changes when you run the same playbook multiple times.

"Run it once, run it again, same result."

2 DESIRED STATE



You define the desired state in your playbook. Ansible checks the current state and only makes changes if needed.

EXAMPLE

If a package is already installed, Ansible will not reinstall it.

3 UNDERSTANDING ANSIBLE RESULTS

ok

Task was successful and no changes were needed.

changed

Task was successful and a change was made.

failed

Task failed. Ansible could not complete the task.

4 RUNNING A PLAYBOOK REPEATEDLY

Run the same playbook multiple times. The first run makes changes. The second run shows mostly "ok" because the system is already in the desired state.

> First run

```
$ ansible-playbook web.yml

TASK [Install nginx] *****
changed: [web01]

TASK [Start nginx] *****
changed: [web01]
```

> Second run

```
$ ansible-playbook web.yml

TASK [Install nginx] *****
ok: [web01]

TASK [Start nginx] *****
ok: [web01]
```

5 IDEMPOTENT MODULES



Most Ansible modules are idempotent. They check the current state before making changes.

EXAMPLES OF IDEMPOTENT MODULES

- package
- service
- user
- file
- copy
- template
- lineinfile
- blockinfile
- unarchive
- get_url

6 NON-IDEMPOTENT SHELL SCRIPTS



Shell scripts and raw commands are usually not idempotent. They run every time, which can cause unintended changes.

EXAMPLE (NOT IDEMPOTENT)

```
- name: Install nginx using shell
  shell: apt-get install -y nginx
```

This command will try to install nginx every time you run the playbook.

USE MODULES INSTEAD

Prefer Ansible modules because they are safer, more reliable and idempotent.

7 WHY IDEMPOTENCY MATTERS IN PRODUCTION



- Prevents configuration drift
- Reduces human error
- Makes automation safe to rerun
- Supports consistent environments
- Enables reliable deployments at scale



JUNIOR ENGINEER TIP

Automate what you understand first. Start small, test often, and build confidence.

8 PRACTICE: RUN THE SAME PLAYBOOK TWICE



- 1 Run your playbook: `ansible-playbook web.yml`
- 2 Run it again: `ansible-playbook web.yml`
- 3 Compare the output (first run: `changed`, second run: `ok`)



YOUR TURN

Create a simple playbook to install and start nginx. Run it twice and observe the difference.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN | PRACTICE | AUTOMATE | GROW

SIMPLE
AUTOMATION
REAL IMPACT

VARIABLES AND FACTS WITHOUT THE CONFUSION

SAME INFRASTRUCTURE
DIFFERENT ENVIRONMENTS
POSSIBILITIES

USE VARIABLES, AUTOMATE SMARTER. KEEP IT FLEXIBLE.

1 VARIABLES



Variables store values that can be reused in your playbooks, inventories and roles. They make your automation flexible and easy to maintain.

"Use variables, not hardcoded values."

2 YAML DATA TYPES

Ansible variables are written in YAML. Common data types:

String	app: nginx
Integer	port: 80
Boolean	enabled: true
List	packages: [nginx, git]
Dictionary	user: name: devops shell: /bin/bash

3 LISTS

A list stores multiple values.

```
packages:
- nginx
- git
- docker
```

4 DICTIONARIES

A dictionary stores key-value pairs.

```
app:
name: myapp
port: 8080
enabled: true
```

5 PLAY VARIABLES

Variables defined directly in a playbook.

```
- hosts: web
vars:
app_name: nginx
port: 80
```

6 INVENTORY VARIABLES

Variables defined in your inventory file.

```
[web]
web1 ansible_host=192.168.1.10 \
app_name=nginx
```

7 host_vars

Variables for a specific host.

```
host_vars/web1.yml
app_name: nginx
port: 80
enabled: true
```

8 group_vars

Variables for a group of hosts.

```
group_vars/web.yml
app_name: nginx
port: 80
enabled: true
```

9 REGISTERED VARIABLES

Store the output of a task in a variable and use it later.

```
- name: Get server uptime
command: uptime
register: uptime_result

- name: Show uptime
debug:
msg: "{{ uptime_result.stdout }}"
```

10 ANSIBLE FACTS

Facts are automatically collected information about the managed node.

```
- name: Show OS information
debug:
msg: "{{ ansible_facts['os_family'] }}"

- name: Show all facts
debug:
var: ansible_facts
```

COMMON FACTS EXAMPLES

```
ansible_facts['os_family'] # Linux
ansible_facts['distribution'] # Ubuntu
ansible_facts['hostname'] # Server hostname
ansible_facts['default_ipv4']['address'] # IP
ansible_facts['memtotal_mb'] # Total memory
ansible_facts['processor'] # CPU info
```

11 VARIABLE PRECEDENCE (BEGINNER MENTAL MODEL)

When the same variable is defined in multiple places, Ansible uses a precedence order. Simplified view:

- 1 Extra variables (-e) (highest)
- 2 Play variables
- 3 Inventory variables (host_vars, group_vars)
- 4 Role variables
- 5 Defaults (lowest)

If a variable is defined in more than one place, the one higher in the list wins.

12 AVOIDING HARDCODED CONFIGURATION



Do not hardcode values like IP addresses, usernames, passwords or file paths in your playbooks.

BAD ❌

```
- name: Install nginx
package:
name: nginx
state: present
when: ansible_host == "192.168.1.10"
```

GOOD ✅

```
- name: Install nginx
package:
name: "{{ app_name }}"
state: present
when: inventory_hostname in
groups['web']
```



JUNIOR ENGINEER TIP

Use variables and facts to make your playbooks reusable, maintainable and environment-independent.

SIMPLE VARIABLES.
POWERFUL AUTOMATION.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CONDITIONS, LOOPS AND REGISTERED RESULTS

BUILD SMARTER PLAYBOOKS THAT REACT TO REAL-WORLD CONDITIONS.

1 WHEN (CONDITIONAL TASKS)

Use when to run a task only if a condition is true.

```
- name: Install nginx only if not installed
  ansible.builtin.package:
    name: nginx
    state: present
  when: nginx_installed is not defined
```



when helps you make your playbooks smarter and more flexible.

2 BOOLEAN CONDITIONS

Use conditions like true, false, and logical operators.

```
- name: Run a task based on a variable
  ansible.builtin.debug:
    msg: "Feature is enabled"
  when: feature_enabled == true
```

COMMON BOOLEAN OPERATORS

== equal to
!= not equal to
and both conditions true
or at least one true
not reverse the condition

3 OPERATING-SYSTEM CONDITIONS

Run tasks based on the target server's operating system.

```
- name: Install httpd on RedHat systems
  ansible.builtin.package:
    name: httpd
    state: present
  when: ansible_facts['os_family'] == 'RedHat'
```

COMMON EXAMPLES

```
ansible_facts['os_family'] == 'RedHat'
ansible_facts['os_family'] == 'Debian'
ansible_facts['distribution'] == 'Ubuntu'
ansible_facts['distribution'] == 'CentOS'
```

4 LOOP (REPEAT A TASK)

Use loop to run a task multiple times.

```
- name: Create multiple users
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
  loop:
    - alice
    - bob
    - charlie
```



loop runs the same task for every item in the list.

5 ITEM (CURRENT LOOP VALUE)

The item variable holds the current value in the loop.

```
- name: Install multiple packages
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - vim
```



item is used inside the task to access the current list value.

6 REGISTERING TASK OUTPUT

Use register to save the output of a task in a variable.

```
- name: Check disk space
  ansible.builtin.command: df -h
  register: disk_output
```



The result is stored in disk_output and can be used in later tasks.

7 USING PREVIOUS TASK RESULTS

You can use registered variables in later tasks.

```
- name: Show disk space output
  ansible.builtin.debug:
    msg: "{{ disk_output.stdout }}"
```



Registered variables contain useful information like stdout, stderr, rc, etc.

8 failed_when (CUSTOM FAILURE)

Use failed_when to define when a task should be marked as failed.

```
- name: Check if a file exists
  ansible.builtin.stat:
    path: /etc/myapp.conf
  register: file_check
  failed_when: file_check.stat.exists == false
```



This task will fail if the file does not exist.

9 changed_when (CUSTOM CHANGE)

Use changed_when to control when a task is marked as changed.

```
- name: Run a command
  ansible.builtin.command: /usr/bin/mycmd
  register: cmd_result
  changed_when: "'completed' in cmd_result.stdout"
```



Use this when a command does not correctly report changes.

10 BUILDING PLAYBOOKS THAT REACT TO SYSTEM STATE



- ✓ Use conditions to run tasks only when needed
- ✓ Loop through lists to handle multiple items
- ✓ Register outputs and use them in later tasks
- ✓ Handle failures and changes properly
- ✓ Make your playbooks idempotent and intelligent



JUNIOR ENGINEER TIP

Always test your conditions and logic with --check mode first. Build playbooks that adapt to the system state, not hardcoded assumptions.

REAL
CONDITIONS
REAL AUTOMATION
REAL IMPACT



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
PRACTICE
AUTOMATE
GROW

TEMPLATES TURN
VARIABLES INTO
REAL
CONFIGURATIONS

TEMPLATES AND CONFIGURATION FILES

DYNAMIC CONFIGURATIONS. CONSISTENT INFRASTRUCTURE. LESS MANUAL WORK.

1 JINJA2



Jinja2

Ansible uses Jinja2, a powerful templating engine, to generate dynamic configuration files using variables, conditions and loops.

- ✓ Simple syntax
- ✓ Flexible and powerful
- ✓ Built into Ansible

2 WHAT A TEMPLATE IS



A template is a text file with variables and logic that Ansible processes to create a final configuration file on the target server.

"One template, many environments."

3 .j2 FILES

Ansible template files usually have a .j2 extension.

```
templates/
  --- nginx.conf.j2
  --- app.conf.j2
  --- database.conf.j2
```

.j2 is just a file extension. It tells you this file contains Jinja2 template code.

4 VARIABLE SUBSTITUTION

Use double curly braces to insert variables into your templates.

```
server {
  listen {{ port }};
  server_name {{ domain }};
}
```

Result (after template is rendered):

```
server {
  listen 80;
  server_name example.com;
}
```

5 CONDITIONS INSIDE TEMPLATES

Use if, elif and else to add logic.

```
{% if enable_ssl %}
  ssl on;
{% else %}
  ssl off;
{% endif %}
```

Example output (enable_ssl = true):

```
ssl on;
```

Example output (enable_ssl = false):

```
ssl off;
```

6 LOOPS INSIDE TEMPLATES

Use for loops to generate repeated content.

```
{% for server in servers %}
  server {{ server }};
{% endfor %}
```

Example (servers list):

```
servers:
- web1
- web2
- web3
```

Generated output:

```
server web1;
server web2;
server web3;
```

7 TEMPLATE MODULE

Use the template module to copy and render a template to the target server.

```
- name: Deploy nginx configuration
  ansible.builtin.template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
```



The template module replaces variables and logic with real values.

8 GENERATING APPLICATION CONFIGURATION

Templates are commonly used for application configuration files.

```
- name: Deploy application config
  ansible.builtin.template:
    src: templates/app.conf.j2
    dest: /etc/myapp/app.conf
```

Used for:

- ✓ Web servers (nginx, apache)
- ✓ Applications (Java, Python, Node.js)
- ✓ Databases (PostgreSQL, MySQL)
- ✓ System configuration files
- ✓ Environment-specific settings

9 VALIDATING CONFIGURATION

Always validate the generated configuration before reloading a service.

```
- name: Validate nginx configuration
  ansible.builtin.command: nginx -t
  register: nginx_test
  changed_when: false
  failed_when: nginx_test.rc != 0
```



Validation helps prevent broken configurations in production.

10 TEMPLATE → SERVER CONFIGURATION → SERVICE

This is the complete flow:



JUNIOR ENGINEER TIP

- ✓ Start with simple templates.
- ✓ Use variables, then add conditions.
- ✓ Test your templates.
- ✓ Validate before restarting services.
- ✓ Keep templates environment-agnostic.

AUTOMATE
CONFIGURE
DEPLOY
SCALE



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

HANDLERS, SERVICES AND CHANGE CONTROL

DETECT
CHANGE
NOTIFY
HANDLE
KEEP SYSTEMS
STABLE

ONLY RESTART WHEN IT'S NECESSARY. THAT'S SMART AUTOMATION.

1 WHAT HANDLERS SOLVE



Handlers solve the problem of running actions (like restarting a service) only when a change actually happens. This prevents unnecessary restarts and keeps your infrastructure stable.

"Make changes, not chaos."

2 NOTIFY



The notify keyword tells Ansible to run a handler when a task makes a change.

```
- name: Deploy configuration file
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx
```

3 HANDLERS



Handlers are defined in a separate handlers section and run only when notified by a task.

```
handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

4 RESTART vs RELOAD

Use reload when the service can apply changes without stopping. Use restart when a full restart is required.

Action	What It Does	When to Use
reload	Reloads configuration	When supported (e.g. nginx, apache)
restart	Stops and starts the service	When reload is not supported or not enough

5 WHY HANDLERS RUN ONLY AFTER CHANGES

- ✓ Handlers run only if the notifying task reports a change.
- ✓ If the task does not change anything, the handler is skipped.
- ✓ This keeps your systems stable and avoids unnecessary restarts.
- ✓ Ansible collects all notifications during the play and runs handlers at the end (unless changed with meta: flush_handlers).

6 MULTIPLE NOTIFICATIONS

Multiple tasks can notify the same handler. The handler will still run only once, even if notified multiple times.

```
- name: Update config file
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/app.conf
  notify: Restart app
- name: Copy application file
  ansible.builtin.copy:
    src: app.py
    dest: /opt/app/app.py
  notify: Restart app
```

- ✓ Even though two tasks notify the same handler, it will run only once.

7 SERVICE MANAGEMENT

Handlers are commonly used to manage services, but they can also run other actions like reloading configurations or restarting applications.

```
handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
- name: Reload apache
  ansible.builtin.service:
    name: apache2
    state: reloaded
```

8 CONFIGURATION CHANGE → HANDLER → SERVICE RELOAD



This flow ensures services are only reloaded or restarted when there is an actual change in configuration.

9 AVOIDING UNNECESSARY PRODUCTION RESTARTS

- ✓ Use handlers to restart services only when needed.
- ✓ Prefer reload if the service supports it.
- ✓ Group related changes and notify the same handler.
- ✓ Test in a non-production environment first.
- ✓ Monitor service health after changes.
- ✓ Avoid using always_run or unnecessary restarts.
- ✓ Keep your playbooks idempotent.



JUNIOR ENGINEER TIP

Use handlers to keep your infrastructure stable. Only restart or reload services when there is a real change. Fewer restarts mean fewer problems.

STABLE SYSTEMS
HAPPY USERS
THAT'S THE GOAL

AUTOMATE
WITH PURPOSE
NOT
DISRUPTION

PRIVILEGE ESCALATION AND SECURE AUTOMATION



AUTOMATE
SECURELY
SCALE
CONFIDENTLY

Run Ansible securely, with the right permissions, the right users, and the right practices.

1 WHAT IS BECOME?



become allows Ansible to run tasks with elevated privileges (usually root) on managed nodes.

```
- hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
```

This runs all tasks as root using sudo on the managed node.

2 BECOME_USER



become_user lets you run tasks as a specific user.

```
- name: Run a task as
  the app user
  become: true
  become_user: appuser
  ansible.builtin.command:
    cmd: whoami
```

Use become_user when a task should not run as root.

3 SUDO



Ansible uses sudo on the managed node by default when become is enabled.

Example:

```
- hosts: servers
  become: true
  become_method: sudo
```

Make sure the remote user has sudo permission.

4 REMOTE USERS



Use a normal, non-root user to connect to servers.

- ✓ Create a dedicated automation user
- ✓ Give only the required permissions
- ✓ Use SSH keys (not passwords)
- ✓ Avoid using root for daily automation

5 SSH KEYS



SSH keys provide secure, passwordless authentication.

- ✓ Generate a key pair
- ✓ Copy the public key to servers
- ✓ Use the key with your inventory
- ✓ Keep your private key safe

```
ssh-copy-id -i ~/.ssh/id_ed25519
user@server-ip
```

6 LEAST PRIVILEGE



Give only the permissions needed to run automation.

- ✓ Do not give full root access
- ✓ Limit sudo to specific commands
- ✓ Use dedicated automation users
- ✓ Reduce the blast radius

"Least privilege reduces risk in production."

7 WHY RUNNING EVERYTHING AS ROOT IS DANGEROUS



- A mistake can damage the system
- Malicious code can do more harm
- Harder to audit what changed
- Increases security risk
- Violates least privilege principles

Run as root only when required, and only for the specific tasks that need it.

8 PROTECTING CREDENTIALS



Never store sensitive data in plain text.

- ✓ Use Ansible Vault for secrets
- ✓ Use environment variables
- ✓ Use external secret managers
- ✓ Restrict access to inventory and playbooks

9 ENVIRONMENT VARIABLES



Use environment variables to avoid hardcoding sensitive information.

```
- name: Use an environment variable
  ansible.builtin.shell: echo $API_TOKEN
  environment:
    API_TOKEN: "{{ lookup('env', 'API_TOKEN') }}"
```

Set variables in your shell or CI/CD environment, not in the playbook.

10 SECURE AUTOMATION PRACTICES



- ✓ Use SSH keys
- ✓ Follow least privilege
- ✓ Use Ansible Vault for secrets
- ✓ Keep playbooks in version control
- ✓ Use code review and testing
- ✓ Log and monitor changes
- ✓ Regularly rotate credentials

Security is part of DevOps. Automate securely from the beginning.

11 SECURITY REMINDER



NEVER HARDCODE PASSWORDS INTO PLAYBOOKS.

Use Ansible Vault, environment variables, or a secret manager instead.

AUTOMATION
BUILDS
BETTER
ENGINEERS



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ANSIBLE VAULT, PROTECT SENSITIVE DATA



SECURE
AUTOMATION
STRONGER
INFRASTRUCTURE

Keep your secrets safe. Automate with confidence.

1 WHY SECRETS REQUIRE SPECIAL HANDLING



- Secrets like passwords, API keys, and tokens can be abused.
- Exposure can lead to data breaches, unauthorized access, and service outages.
- Configuration files and playbooks are often stored in Git, so secrets must be protected.
- Compliance and security policies require proper secret management.

2 WHAT ANSIBLE VAULT PROVIDES



- ✓ Encrypts sensitive data in files.
- ✓ Keeps secrets in your repository, but encrypted.
- ✓ Integrates with Ansible workflows.
- ✓ Simple to use with the ansible-vault CLI.
- ✓ Helps you follow security best practices.

Ansible Vault does not hide secrets from users who have the decryption password.

3 ANSIBLE-VAULT CREATE



Create a new encrypted file.

```
ansible-vault create secrets.yml
```

You will be prompted for a vault password.

4 ENCRYPT AN EXISTING FILE



Encrypt an existing file.

```
ansible-vault encrypt secrets.yml
```

The file will be encrypted and can no longer be read in plain text.

5 DECRYPT A FILE



Decrypt an encrypted file.

```
ansible-vault decrypt secrets.yml
```

The file will be restored to plain text.

6 EDIT AN ENCRYPTED FILE



Edit an encrypted file directly.

```
ansible-vault edit secrets.yml
```

The file is decrypted in a temporary location, you edit it, then it is encrypted again automatically.

7 VIEW AN ENCRYPTED FILE



View the contents without decrypting permanently.

```
ansible-vault view secrets.yml
```

Useful for quick checks.

8 VAULT IDS



Use different passwords for different environments.

```
ansible-playbook site.yml \
  --vault-id dev@dev.vault \
  --vault-id prod@prod.vault
```

Helps manage multiple encrypted files with different passwords.

9 VAULT PASSWORD FILES



Store the vault password in a file (safer for automation).

```
ansible-playbook site.yml \
  --vault-password-file ~/.vault_pass.txt
```

Keep the password file secure and out of version control.

10 CI/CD CONSIDERATIONS



- ✓ Use vault password files or CI/CD secrets.
- ✓ Inject the password as an environment variable.
- ✓ Do not hardcode passwords in pipelines.
- ✓ Use separate vault IDs for different environments.
- ✓ Ensure your CI/CD logs do not expose secrets.

11 WHAT VAULT DOES NOT PROTECT



- ✗ It does not protect variables that are decrypted during runtime.
- ✗ It does not protect secrets in shell history or logs.
- ✗ It does not encrypt data outside of Ansible files (e.g. cloud provider secrets).
- ✗ It does not replace external secret managers for large-scale environments.

12 WHEN TO USE EXTERNAL SECRET MANAGERS



- ✓ When you need dynamic secret rotation.
- ✓ When managing secrets across multiple tools and teams.
- ✓ In large, complex, or multi-cloud environments.
- ✓ Examples: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager.



SECURITY REMINDER

NEVER HARDCODE PASSWORDS INTO PLAYBOOKS.

Use Ansible Vault, environment variables, or an external secret manager instead.

PROTECT
YOUR SECRETS
PROTECT
YOUR FUTURE

ROLES

STOP BUILDING GIANT PLAYBOOKS

ORGANIZE
REUSE
AUTOMATE
AT SCALE

MODULAR. REUSABLE. SCALABLE. REAL AUTOMATION.

1

WHY LARGE PLAYBOOKS BECOME DIFFICULT



- Hard to read and maintain
- Difficult to reuse code
- Prone to errors
- Not scalable for teams
- Changes can break other parts
- Becomes messy over time

"If your playbook is doing everything, it's time to use roles."

2

WHAT AN ANSIBLE ROLE IS

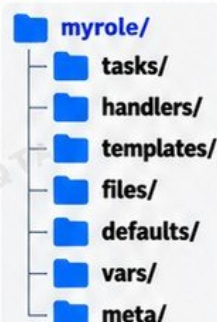


A role is a structured way to organize Ansible content (playbooks, tasks, templates, files, variables, handlers, etc.) into a reusable and modular format.

- ✓ Encourages best practices
- ✓ Reusable across projects
- ✓ Easier collaboration
- ✓ Standard structure
- ✓ Scales for real-world infrastructure

3

ROLE DIRECTORY STRUCTURE



This is the standard directory structure created by ansible-galaxy role init.

4

tasks/



Contains the main tasks that define what the role does.

```
# tasks/main.yml
- name: Install nginx
  ansible.builtin.package:
    name: nginx
    state: present
```

5

handlers/



Contains handlers that run when notified (e.g. service restart).

```
# handlers/main.yml
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

6

templates/



Stores Jinja2 templates for configuration files.

```
# templates/nginx.conf.j2
server {
  listen 80;
  server_name {{ domain }};
}
```

7

files/



Stores static files to be copied to the target hosts.

```
files/
├── nginx.conf
└── app.tar.gz
```

8

defaults/



Contains default variables with low precedence.

```
# defaults/main.yml
nginx_port: 80
app_user: www-data
```

9

vars/



Contains role variables with higher precedence than defaults.

```
# vars/main.yml
nginx_package: nginx
nginx_service: nginx
```

10

meta/



Contains metadata about the role.

```
# meta/main.yml
galaxy_info:
  author: veriqta
  description: Nginx role
  license: MIT
```

11

ansible-galaxy role init



Creates a new role with the standard structure.

```
ansible-galaxy role init myrole
```

This command generates the full directory structure for you.

12

REUSABLE INFRASTRUCTURE AUTOMATION



- ✓ Write once, use many times
- ✓ Share roles across teams
- ✓ Use in multiple environments
- ✓ Combine roles for complex setups
- ✓ Leverage community roles (Ansible Galaxy)

13

ROLE DESIGN FOR TEAMS



- ✓ Keep roles focused and single-purpose
- ✓ Use clear and consistent naming
- ✓ Document variables and usage
- ✓ Follow Ansible best practices
- ✓ Use version control (Git)
- ✓ Review roles before production
- ✓ Encourage reuse and contribution



KEY TAKEAWAY

"Roles make your Ansible automation cleaner, reusable, and ready for scale."

COLLECTIONS AND ANSIBLE GALAXY



DISCOVER
USE
EXTEND
AUTOMATE

MORE MODULES. MORE ROLES. MORE POSSIBILITIES.

1 WHAT COLLECTIONS ARE



- ✓ Collections are a way to package and distribute Ansible content (modules, plugins, roles, etc.).
- ✓ They make it easy to share, reuse and manage automation at scale.
- ✓ Collections can be created by the community, vendors or your own team.

2 WHAT A COLLECTION CAN INCLUDE



MODULES
Add new functionality



PLUGINS
Extend Ansible
(e.g. inventory, lookup, callback)



ROLES
Reusable infrastructure automation



OTHER CONTENT
Playbooks, docs, tests, etc.

3 NAMESPACES



- ✓ Collections are organized using namespaces.
- ✓ The namespace usually represents a person, organization or vendor.
- ✓ Examples: ansible, community, redhat, aws, azure, google

"Namespace + Collection Name = Unique Identifier"

4 FULLY QUALIFIED COLLECTION NAMES

`namespace.collection.module`

- ✓ This is called a Fully Qualified Collection Name (FQCN).
- ✓ It ensures you use the correct module.
- ✓ Example: ansible.builtin.copy
- ✓ Always prefer the FQCN to avoid conflicts.

5 INSTALL A COLLECTION



Install a collection from Ansible Galaxy.

```
ansible-galaxy collection install community.general
```

- ✓ Downloads and installs the collection into your Ansible environment.
- ✓ You can install multiple collections at the same time.

6 REQUIREMENTS.YML



- ✓ Use a requirements.yml file to manage collections for your project.

```
# requirements.yml
collections:
- name: community.general
  version: ">=8.0.0"
- name: amazon.aws
  version: "5.0.0"
```

- ✓ Install all collections with:
`ansible-galaxy collection install -r requirements.yml`

7 PINNING VERSIONS



- ✓ Pinning versions ensures stability and consistent environments.
- ✓ Use specific versions or version ranges in requirements.yml.
- ✓ Helps prevent breaking changes when new versions are released.

```
- name: community.general
  version: "8.0.0"
- name: amazon.aws
  version: ">=5.0.0, <6.0.0"
```

8 COMMUNITY vs VENDOR COLLECTIONS



COMMUNITY COLLECTIONS

- ✓ Built and maintained by the open source community
- ✓ Wide range of tools and integrations
- ✓ Great for general use
- ✓ Examples: community.general, community.aws



VENDOR COLLECTIONS

- ✓ Built and maintained by technology vendors (e.g. Red Hat, AWS, Azure)
- ✓ Often more stable and officially supported
- ✓ Best for vendor-specific tools and services
- ✓ Examples: redhat.rhel, amazon.aws, azure.azcollection

9 REVIEW BEFORE YOU TRUST



- ✓ Always review third-party collections before using them.
- ✓ Check the source (GitHub)
- ✓ Review documentation
- ✓ Look at recent activity and maintenance
- ✓ Check for known issues
- ✓ Understand what the collection does on your systems
- ✓ Use trusted sources and official namespaces when possible

10 KEY TAKEAWAYS



- ✓ Collections make Ansible more powerful and extensible.
- ✓ Use FQCN to avoid conflicts.
- ✓ Manage collections with requirements.yml.
- ✓ Pin versions for stability.
- ✓ Use community and vendor collections appropriately.
- ✓ Always review third-party automation before trusting it.

“GOOD AUTOMATION USES THE RIGHT TOOLS FROM TRUSTED SOURCES.”

**COLLABORATE
AUTOMATE
SCALE
TOGETHER**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

TROUBLESHOOTING ANSIBLE LIKE AN ENGINEER



FIND
FIX
LEARN
IMPROVE

SAME PROBLEMS. BETTER SOLUTIONS. STRONGER ENGINEERS.

1 READ THE FAILURE FIRST



- ✓ Read the full error message.
- ✓ Identify where it failed.
- ✓ Look for the actual cause (not just the last line).
- ✓ Understand what Ansible was trying to do.

"The error message is telling you a story. Read it."

2 UNREACHABLE vs FAILED

UNREACHABLE

- Ansible cannot connect to the host.
- Usually a network, SSH or authentication issue.
- Tasks do not run.

```
UNREACHABLE! => {
  "msg": "Failed to connect
to the host via ssh"
}
```

FAILED

- Ansible connected to the host, but the task failed.
- A configuration, module or command issue.
- Tasks ran but returned an error.

```
FAILED! => {
  "msg": "The task
failed ..."
}
```

3 SSH AUTHENTICATION FAILURES



- ✓ Check username and SSH keys.
- ✓ Verify the key is added to the server.
- ✓ Ensure correct permissions (600 for private key).
- ✓ Test manual SSH login first.

```
ssh -i ~/.ssh/id_rsa user@server-ip
```

4 PERMISSION DENIED



- ✓ User does not have the required privileges.
- ✓ Use become: true for privileged tasks.
- ✓ Verify sudo permissions.
- ✓ Check file and directory permissions.

```
- name: Install package
  become: true
  ansible.builtin.package:
    name: nginx
    state: present
```

5 MISSING PYTHON



- ✓ Ansible requires Python on managed Linux hosts.
- ✓ Install Python using raw module if needed.

```
- name: Install Python
  ansible.builtin.raw: |
    sudo apt-get update &&
    sudo apt-get install -y python3
```

6 UNDEFINED VARIABLES



- ✓ Variable is not defined or out of scope.
- ✓ Check variable names, spelling and precedence.
- ✓ Use default values when appropriate.

```
- debug:
  msg: "{{ my_variable }}"
  when: my_variable is defined
```

7 YAML SYNTAX ERRORS



- ✓ Check indentation (spaces, not tabs).
- ✓ Verify colons and quotes.
- ✓ Use a YAML linter or syntax check.

```
- name: Install nginx
  package:
    name nginx # Missing colon
    state: present
```

8 MISSING MODULES OR COLLECTIONS



- ✓ The module or collection is not installed.
- ✓ Check the module name and FQCN.
- ✓ Install the required collection.

```
ansible-galaxy collection install
community.general
```

Use Fully Qualified Collection Names (FQCN) to avoid conflicts.

9 -v, -vv, -vvv (INCREASE VERBOSITY)



- ✓ -v shows more details.
- ✓ -vv shows even more details.
- ✓ -vvv shows full debugging information.

```
ansible-playbook site.yml -vvv
```

Use higher verbosity when you need to see what Ansible is actually doing.

10 USEFUL COMMAND OPTIONS



- ✓ --syntax-check Check playbook syntax without running it.
- ✓ --check Run in check mode (dry run).
- ✓ --diff Show the changes that would be made.

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --check --diff
```

11 TROUBLESHOOTING MODEL



12 FINAL TIP



- ✓ Be systematic.
- ✓ Fix one issue at a time.
- ✓ Test as you go.
- ✓ Learn from every error.
- ✓ Better troubleshooting makes you a stronger engineer.

ANSIBLE IN CI/CD AND INFRASTRUCTURE WORKFLOWS



AUTOMATE
TEST
DEPLOY
REPEAT

Automate everything. From code to infrastructure.

1 GIT REPOSITORY



- ✓ Store playbooks, roles and configuration in version control.
- ✓ Track changes and collaborate with your team.
- ✓ Use branches and pull requests for safe changes.

```
git add .
git commit -m "Update playbook"
git push
```

2 PULL REQUEST



- ✓ Open a pull request for changes.
- ✓ Get code review and feedback.
- ✓ Run automated checks before merging.

```
git checkout -b feature/ansible
git push origin feature/ansible
```

No review. No merge. No deploy.

3 PLAYBOOK VALIDATION



- ✓ Check your playbooks before running them.
- ✓ Catch errors early.
- ✓ Use `ansible-playbook --syntax-check`.

```
ansible-playbook playbook.yml --syntax-check
```

Validation saves time and prevents failures in production.

4 LINTING



- ✓ Enforce best practices.
- ✓ Find style and syntax issues.
- ✓ Keep your playbooks clean and consistent.

```
ansible-lint playbook.yml
```

Clean code. Fewer problems.

5 TESTING



- ✓ Use test environments.
- ✓ Verify changes work as expected.
- ✓ Run tests in CI pipeline.

```
ansible-playbook playbook.yml
--check --diff
```

Test early. Deploy with confidence.

6 CI PIPELINE



- ✓ Automate validation, linting, testing and deployment.
- ✓ Run on every push or PR.
- ✓ Fail fast on errors.



7 DEPLOYMENT AUTOMATION



- ✓ Automate releases with Ansible.
- ✓ Keep environments consistent.
- ✓ Reduce manual work and errors.

```
ansible-playbook deploy.yml -i inventory/
```

Automate more. Click less.

8 ANSIBLE WITH TERRAFORM



Terraform

Ansible

- ✓ Terraform provisions infrastructure.
- ✓ Ansible configures systems.
- ✓ Together they deliver complete environments.

Terraform
(Infrastructure)



Ansible
(Configuration)

9 ANSIBLE IN JENKINS, GITHUB ACTIONS AND GITLAB CI



Jenkins



GitHub
Actions



GitLab
CI

- ✓ Run playbooks as part of your pipeline.
- ✓ Use credentials securely.
- ✓ Automate across multiple platforms.

```
ansible-playbook site.yml
```

10 WHY INFRASTRUCTURE AUTOMATION BELONGS IN VERSION CONTROL



- ✓ Tracks changes and history.
- ✓ Enables collaboration and code review.
- ✓ Provides rollback when needed.
- ✓ Keeps your infrastructure reproducible.

“INFRASTRUCTURE AS CODE
IS NOT JUST A PRACTICE,
IT'S A SAFETY NET.”

SAME CODE.
SAME ENVIRONMENT.
EVERY TIME.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PRODUCTION ANSIBLE RUN AUTOMATION WITHOUT FEAR

AUTOMATE
SECURE
SCALE
SUCCEED

Plan. Protect. Deploy. Verify. Repeat.

1 DEVELOPMENT, STAGING AND PRODUCTION INVENTORIES

-  dev
 - Keep separate inventories for each environment.
-  staging
 - Use group variables for environment-specific settings.
-  prod
 - Prevent accidental changes to production.

2 LIMITING TARGET HOSTS



Use `--limit` to run playbooks on specific hosts or groups.

```
ansible-playbook site.yml \
--limit webserver
```

Reduces risk and helps you make controlled changes.

3 CHECK MODE



Preview changes without making any changes.

```
ansible-playbook site.yml \
--check
```

Useful for testing and change reviews.

4 DIFF MODE



Shows the exact changes that would be made.

```
ansible-playbook site.yml \
--check --diff
```

Helps you review changes before applying them.

5 SERIAL EXECUTION



Use serial to update servers in batches.

```
serial: 2
```

Updates a few servers at a time to reduce risk.

6 ROLLING CHANGES



- Update gradually
- Keep the service available
- Combine with serial
- Monitor between batches

Minimizes downtime in production.

7 TAGS



Use tags to run specific tasks.

```
ansible-playbook site.yml \
--tags "nginx"
```

Run only what you need.

8 VERSION CONTROL



- Keep playbooks in Git.
- Track changes over time.
- Use meaningful commit messages.
- Use branches for new changes.
- Maintain a clean project structure.

9 PEER REVIEW



- Review playbooks before merging.
- Catch mistakes early.
- Ensure best practices.
- Improve team knowledge.
- Use pull requests.

10 BACKUPS



- Back up critical data and configuration.
- Automate backups where possible.
- Test your restore process.
- Never run destructive changes without a backup plan.

11 FAILURE HANDLING



- Use error handling options.
- Control failures with `failed_when` and `ignore_errors`.
- Check task results.
- Stop when something critical fails.
- Investigate and fix the root cause.

12 ROLLBACK PLANNING



- Have a rollback plan before making changes.
- Use version control to revert.
- Keep previous configurations.
- Test rollback procedures.
- Document rollback steps.

13 AVOIDING CONFIGURATION DRIFT



- Use Ansible for all configuration changes.
- Avoid manual changes on servers.
- Run playbooks regularly.
- Monitor system state.
- Keep inventories and variables updated.

14 PRODUCTION VERIFICATION



- Verify the application is working.
- Check services, logs and health.
- Test from the end user perspective.
- Monitor after deployment.
- Do not stop at a successful playbook run.

15 A SUCCESSFUL ANSIBLE RECAP DOES NOT GUARANTEE A HEALTHY APPLICATION



```
PLAY RECAP *****
web01 : ok=5  changed=2  unreachable=0  failed=0
web02 : ok=5  changed=2  unreachable=0  failed=0
```



- Ansible only ensures the tasks ran successfully.
- The application might still be misconfigured.
- Always verify the actual service, logs and functionality in production.
- Automation success is not the same as business success.



JUNIOR ENGINEER TIP

Make small, safe changes. Use check mode, review your playbooks, and always verify after deployment.

AUTOMATION
CREATES
FREEDOM

FINAL PRODUCTION LAB, CONFIGURE AND RECOVER SERVERS

Put everything together. Deploy. Break. Fix. Learn. Be Ready for Production.

AUTOMATE
DEPLOY
SCALE
RECOVER



LAB STEPS

- 1 Build an Ansible project
- 2 Create inventory groups
- 3 Configure SSH access
- 4 Test connectivity
- 5 Create reusable variables
- 6 Install a web server
- 7 Deploy a configuration template
- 8 Deploy application content
- 9 Configure and start the service
- 10 Use a handler for reload/restart
- 11 Protect sensitive variables
- 12 Validate with check mode
- 13 Run the production playbook
- 14 Verify the service
- 15 Introduce a deliberately broken configuration
- 16 Diagnose the failure
- 17 Restore the correct configuration
- 18 Rerun the playbook
- 19 Verify recovery
- 20 Run again and confirm idempotency



SAMPLE COMMANDS YOU WILL USE

```
# Run the playbook
ansible-playbook site.yml

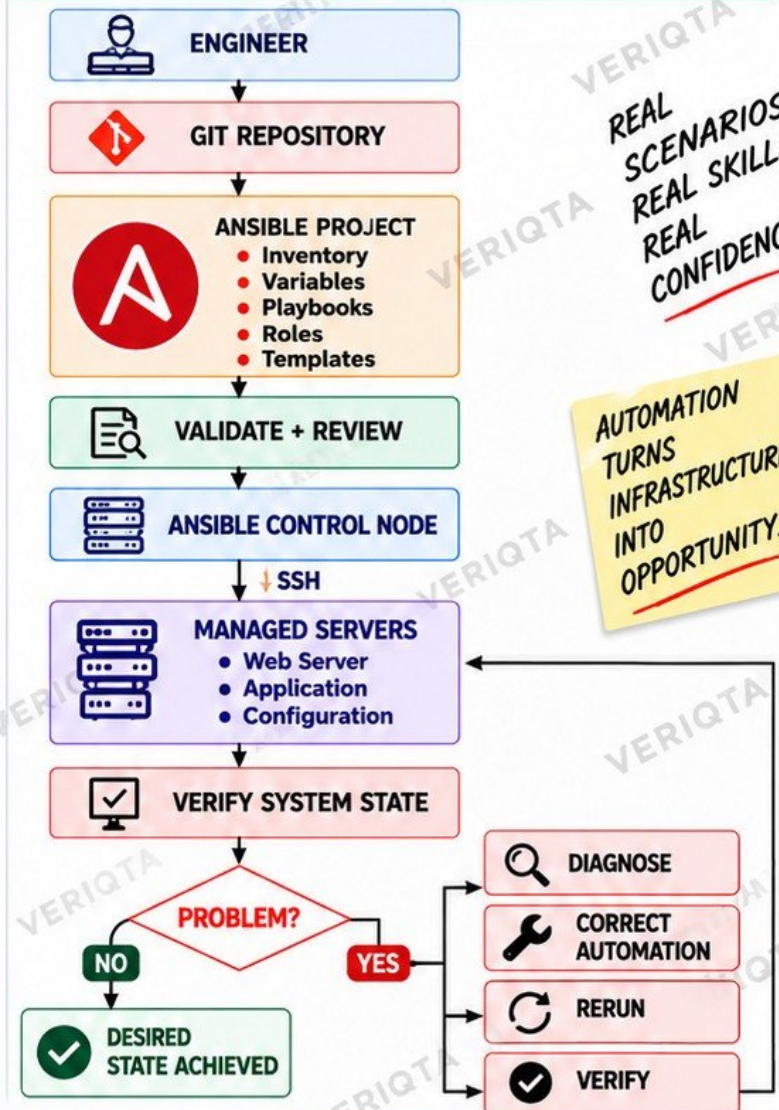
# Check mode (dry run)
ansible-playbook site.yml --check --diff

# Limit to specific hosts
ansible-playbook site.yml --limit webservers

# Run with tags
ansible-playbook site.yml --tags "web"

# Increase verbosity (for troubleshooting)
ansible-playbook site.yml -vvv
```

FINAL ARCHITECTURE



REAL
SCENARIOS.
REAL SKILLS.
REAL
CONFIDENCE.

AUTOMATION
TURNS
INFRASTRUCTURE
INTO
OPPORTUNITY.



JUNIOR ENGINEER TIP

It is normal to break things in a lab. What matters is that you can diagnose, fix and recover them. That is a real production skill.



REMEMBER

A successful Ansible recap does not guarantee a healthy application. Always verify the actual service, logs and functionality.

BUILD
PRACTICE
IMPROVE
GROW

SAME
KNOWLEDGE
BIGGER
OPPORTUNITIES